

UNIVERSIDADE
FEDERAL DO CEARÁ

Test Smells em projetos OpenSource

Projeto: Microsoft - APM
Fork: https://github.com/victtorqueiroz/apm
Ferramenta de detecção: TEMPY
Métricas: Radon e pytest
Uso de IA: Claude (orientação e revisão) e Claude Code com Claude Sonnet 5.5 e Claude Opus 5.5 (edição dos testes e commits).
Test Smells: Exception Handling, Verbose Test, Unknown Test e Conditional Test Logic

Disciplina: Qualidade de Software
Equipe: Victor Emanuel Silva de Queiroz - 567683 e Radlei Eugenio Doroth - 568379
Professora: Carla Ilane Moreira Bezerra
Monitor: Gabriel Alves Pessoa

UNIVERSIDADE FEDERAL DO CEARÁ

Sumário

1. Exception Handling.....2
2. Verbose Test.....8
3. Unknown Test.....12
4. Conditional Test Logic.....19

1. Exception Handling

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
- **Exception Handling (commit b48a0fc)**
- em tests/unit/test_mcp_overlays.py, método test_validate_stdio_shell_string_error_includes_fix_it (linha 146).

Minhas principais dificuldades na remoção do code smell são:
- Entender o fluxo de try/except/else e o funcionamento do with pytest.raises. Testei o conceito num arquivo à parte, provocando falhas de propósito.
- Decidir onde ficam os asserts sobre a mensagem: dentro do bloco with eles nunca seriam executados, então precisam ficar depois dele.

Usei o CATSPY para:
- Entender a definição do smell (exceção tratada manualmente com try/except em vez dos recursos do framework) e aplicar a refatoração sugerida na página do Exception Handling (assertRaises/pytest.raises).

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Substituir try/except/else + pytest.fail por with pytest.raises(ValueError) as exc_info, verificando a mensagem com str(exc_info.value) após o bloco.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
- **Exception Handling (commit 9dd11ec)**
- em tests/unit/compilation/test_compilation.py, método test_validation_error_display (linha 424).

Minhas principais dificuldades na remoção do code smell são:

- O teste verifica que a função não lança exceção, ou seja, não há exceção esperada para usar com pytest.raises.
- Após a refatoração, o teste fica sem nenhuma asserção e o TEMPY passa a marcá-lo como Unknown Test, como já era antes, porque fica sem assert. A função só imprime na tela e não retorna valor útil para verificar.

Usei o CATSPY para:

- Confirmar que o padrão try/except + fail caracteriza o smell e adaptar a refatoração ao caso “não deve lançar exceção”.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Remover o try/except Exception + self.fail e chamar a função diretamente: uma exceção não capturada já reprova o teste no pytest e ainda preserva o traceback original.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Exception Handling (commit b71eb00)**
- em tests/unit/test_build_spec.py, método test_spec_file_compiles_without_syntax_errors (linha 125).

Minhas principais dificuldades na remoção do code smell são:

- A mesma situação do caso anterior: o teste verifica ausência de exceção. Ao remover o try, perde-se apenas a mensagem customizada do pytest.fail.
- O método já era marcado como Unknown Test antes da refatoração e continua sendo.

Usei o CATSPY para:

- Reaplicar a refatoração da página do Exception Handling, usando o mesmo padrão já adaptado no caso anterior.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Remover o try/except SyntaxError + pytest.fail e chamar compile() diretamente.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Exception Handling (commit e348537)**

- em tests/unit/test_runtime_detection.py, método test_full_detection_workflow (linha 197).

Minhas principais dificuldades na remoção do code smell são:
- O try/except está dentro de um for com self.subTest. Foi preciso remover só o tratamento de exceção, sem alterar o laço, que é outro smell (Conditional Test Logic) tratado pela minha dupla.

Usei o CATSPY para:
- Reaplicar a refatoração da página do Exception Handling, usando o mesmo padrão já adaptado nos casos anteriores.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Remover o try/except ValueError + self.fail, mantendo a chamada e o self.assertIsNotNone(client) dentro do subTest.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
- **Exception Handling (commit f322392)**
- em tests/unit/test_script_runner.py, método test_resolve_prompt_file_local_exists (linha 353).

Minhas principais dificuldades na remoção do code smell são:
- O try aqui não verifica exceção: é um try/finally usado só para restaurar o diretório de trabalho. O TEMPY marca qualquer bloco try, enquanto o CATSPY descreve o uso de try/except para verificar exceções .
- Executado sozinho, o arquivo gera ImportError por importação circular no código-fonte (problema pré-existente). Contornei executando-o junto com outro arquivo de teste.

Usei o CATSPY para:
- Comparar a definição do catálogo com a regra de detecção do TEMPY e identificar que este caso é uma variação do smell (try usado para limpeza, não para verificar exceção).

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Substituir tempfile.TemporaryDirectory() + os.chdir + try/finally pelas fixtures tmp_path e monkeypatch.chdir(tmp_path), que restauram o diretório automaticamente ao fim do teste, mesmo em caso de falha.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Exception Handling (commit 573c429)**
- em tests/unit/test_script_runner.py, método test_resolve_prompt_file_dependency_root (linha 371).

Minhas principais dificuldades na remoção do code smell são:
- A mesma limitação do caso anterior: try/finally usado só para restaurar o diretório. Foi preciso reindentar o corpo do método após remover dois níveis de bloco.

Usei o CATSPY para:
- Reaplicar a refatoração usando o mesmo padrão já adaptado no caso anterior.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Substituir TemporaryDirectory + try/finally + os.chdir por tmp_path e monkeypatch.chdir(tmp_path).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
- **Exception Handling (commit 1ad3a2f)**
- em tests/unit/test_script_runner.py, método test_resolve_prompt_file_dependency_subdirectory (linha 393).

Minhas principais dificuldades na remoção do code smell são:
- A mesma limitação do caso anterior.

Usei o CATSPY para:
- Reaplicar a refatoração usando o mesmo padrão já adaptado no caso anterior.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Substituir TemporaryDirectory + try/finally + os.chdir por tmp_path e monkeypatch.chdir(tmp_path).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
- **Exception Handling (commit 8ae1373)**
- em tests/unit/test_script_runner.py, método test_resolve_prompt_file_multiple_dependencies (linha 417).

Minhas principais dificuldades na remoção do code smell são:
- A mesma limitação do caso anterior. Os caminhos relativos usados no teste continuam válidos porque o diretório atual passa a ser o tmp_path.

Usei o CATSPY para:

- Reaplicar a refatoração usando o mesmo padrão já adaptado no caso anterior.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Substituir TemporaryDirectory + try/finally + os.chdir por tmp_path e monkeypatch.chdir(tmp_path).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Exception Handling (commit df2cc77)**
- em tests/unit/test_script_runner.py, método test_resolve_prompt_file_no_apm_modules (linha 446).

Minhas principais dificuldades na remoção do code smell são:

- O método já usava pytest.raises dentro do try/finally. Foi preciso distinguir os dois: o pytest.raises é a forma correta e foi mantido, e só o try/finally foi removido.

Usei o CATSPY para:

- Confirmar que o pytest.raises já existente é a forma recomendada pelo catálogo e reaplicar o padrão dos casos anteriores para o try/finally.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Substituir TemporaryDirectory + try/finally + os.chdir por tmp_path e monkeypatch.chdir(tmp_path), mantendo o with pytest.raises(FileNotFoundError).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Exception Handling (commit dd1d4f0)**
- em tests/unit/test_config_command.py, método test_config_show_outside_project (linha 32).

Minhas principais dificuldades na remoção do code smell são:

- O teste voltava ao diretório original antes do assert porque, no Windows, uma pasta temporária não pode ser apagada enquanto é o diretório atual. Com tmp_path, o pytest cuida disso.
- Ao rodar o arquivo, 3 testes de TestCoworkSkillsDirFunctions falham no Windows por usarem caminhos POSIX (/absolute/...). A falha já existia antes e não tem relação com a refatoração.

Usei o CATSPY para:

- Reaplicar a refatoração usando o mesmo padrão já adaptado nos casos de try/finally.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Substituir TemporaryDirectory + try/finally + os.chdir por tmp_path e monkeypatch.chdir(tmp_path), mantendo o patch de get_version.

UNIVERSIDADE FEDERAL DO CEARÁ

2. Verbose Test

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Verbose Test (commit 69748b6)**
- em tests/unit/integration/test_skill_integrator_hermetic.py, método test_collision_warning_with_logger (linha 572).

Minhas principais dificuldades na remoção do code smell são:

- Entender o que separar: o teste verifica uma única coisa (o aviso de colisão), mas gastava mais da metade das linhas montando o cenário. Tive que identificar quais variáveis da preparação eram usadas depois, para que a função auxiliar as devolvesse.
- Tive que lembrar e pesquisar o padrão AAA(Arrange, Act, Assert): preparar, executar e verificar. Foi isso que me ajudou a decidir o que ia pra função auxiliar e o que ficava no teste.

Usei o CATSPY para:

- Entender a definição do smell (método de teste com mais de 30 linhas ou com múltiplas responsabilidades) e comparar com a regra do TEMPY, que marca métodos com 45 linhas ou mais, contando comentários e linhas em branco.
- Concluir que a refatoração sugerida no catálogo (dividir o teste) não se aplicava, porque o teste tem uma única responsabilidade.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extract Method: a preparação do cenário (pacote, skill pré-existente e dependência de outro dono) foi movida para a função auxiliar _make_colliding_skill, reaproveitando o parâmetro dep_ref da auxiliar existente _make_package_info.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Verbose Test (commit 17ab795)**
- em tests/unit/integration/test_skill_integrator_phase3w4.py, método test_collision_warning_with_logger (linha 612).

Minhas principais dificuldades na remoção do code smell são:

- Percebi que o teste é uma cópia idêntica do caso anterior, em outro arquivo (código duplicado entre arquivos de teste). Remover essa duplicação está fora do escopo deste smell.

Usei o CATSPY para:
- Reaplicar a refatoração usando o mesmo padrão já adaptado no caso anterior.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Extract Method: a mesma função auxiliar _make_colliding_skill, criada neste arquivo, dessa forma, consegui reaproveitar tal função.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
- **Verbose Test (commit bdba062)**
- em tests/unit/integration/test_skill_integrator.py, método test_native_skill_preserves_complete_structure (linha 1434).

Minhas principais dificuldades na remoção do code smell são:
- A auxiliar precisou ser um método da classe (usando self.project_root), seguindo o padrão da classe, que já tinha o método _create_package_info.
- O texto do SKILL.md (string de várias linhas) precisou ficar sem recuo, senão o conteúdo do arquivo mudaria.

Usei o CATSPY para:
- Confirmar que o teste tem uma única responsabilidade (preservar a estrutura de pastas) e que o excesso de linhas vinha da preparação.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Extract Method: a criação do pacote de exemplo (SKILL.md, scripts/, references/ e assets/) foi movida para o método _create_complete_skill_package.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
- **Verbose Test (commit 7c77500)**
- em tests/unit/integration/test_skill_integrator.py, método test_force_overwrites_identical_bin (linha 4582).

Minhas principais dificuldades na remoção do code smell são:
- O teste executa duas etapas (sem force e com force), o que se encaixa no critério de “múltiplas responsabilidades” do CATSPY. Decidi não dividir o teste porque a segunda etapa depende do estado deixado pela primeira, e dividir mudaria o que ele verifica.
- Entender que a classe já tinha uma auxiliar (_make_plugin_package) que cria só o objeto do pacote, e que a nova auxiliar deveria usá-la internamente.

Usei o CATSPY para:
- Avaliar se a refatoração sugerida (dividir o teste em métodos menores) era adequada. Concluí que não, neste caso, pelo motivo acima.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extract Method: a criação da pasta do usuário, do plugin com bin/ e da cópia já instalada foi movida para _make_preinstalled_plugin, que devolve project_root, pkg_info e dest_bin.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Verbose Test (commit aa05422)**
- em tests/unit/deps/test_shared_clone_cache.py, método test_short_sha_skips_tier1_and_resolves_via_tier2 (linha 692).

Minhas principais dificuldades na remoção do code smell são:

- Neste teste, a execução só funciona dentro de um patch (git falso), então a auxiliar precisou conter preparação e execução, devolvendo a lista de comandos capturados.
- Na primeira tentativa, não apaguei a preparação antiga do teste. Na segunda, apaguei junto a chamada da auxiliar. Os dois erros foram detectados pelo pytest (NameError).

Usei o CATSPY para:

- Reaplicar a ideia de manter no teste apenas o que explica o que o teste confere, deixando os detalhes técnicos fora dele.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extract Method: a montagem do downloader, do git falso e a chamada de _bare_clone_with_fallback foram movidas para _run_bare_clone_resolving, que recebe a ref e o SHA resolvido e devolve os comandos executados.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Verbose Test (commit 4fe7174)**
- em tests/unit/integration/test_hook_integrator.py, método test_stale_sidecar_removed_on_sync (linha 1047).

Minhas principais dificuldades na remoção do code smell são:

- Perceber que o excesso não vinha de uma preparação complexa, mas de repetição: o mesmo dicionário de hook estava escrito duas vezes, em formato expandido.
- O teste contém um if (Conditional Test Logic), que mantive porque é outro smell, tratado pela minha dupla.

Usei o CATSPY para:

- Confirmar que o teste tem uma responsabilidade única e que a redução de tamanho não deveria alterar os dados nem as verificações.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extract Variable (remoção de duplicação): o hook foi definido uma única vez numa variável e reaproveitado no settings.json e no sidecar.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Verbose Test (commit a8ddd23)**
- em tests/unit/integration/test_hook_integrator_defect_regression.py, método test_unambiguous_legacy_leaf_marker_migrates_to_canonical_owner (linha 744).

Minhas principais dificuldades na remoção do code smell são:

- Decidir o que extrair: a preparação misturava o pacote instalado, o lockfile e o estado legado. Extraí só o pacote e o lockfile. O estado legado ficou no teste, por ser o cenário que ele quer mostrar.
- O teste ficou com cerca de 32 linhas: abaixo do limite do TEMPY (45), mas acima do limite do CATSPY (30).

Usei o CATSPY para:

- Comparar o limite de 30 linhas do catálogo com o resultado obtido e justificar por que não reduzi mais.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extract Method: a criação do pacote org-a/hooks, do lockfile e do PackageInfo foi movida para a função _install_org_a_hooks_package.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Verbose Test (commit 2516300)**
- em tests/unit/deps/test_shared_clone_cache.py, método test_bare_action_rmtrees_target_before_init (linha 1038).

Minhas principais dificuldades na remoção do code smell são:

- Percebi que a montagem do downloader (23 linhas) estava copiada do método _make_downloader de outra classe do mesmo arquivo. Reaproveitar aquele método exigiria alterar outra classe, fora do escopo.

Usei o CATSPY para:

- Reaplicar a refatoração usando o mesmo padrão dos casos anteriores.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extract Method: a construção do downloader com o plano de transporte foi movida para o método _make_downloader desta classe.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
- **Verbose Test (commit 18ab66f)**
- em tests/unit/deps/test_shared_clone_cache.py, método test_fetch_action_uses_explicit_url_not_origin (linha 1481).

Minhas principais dificuldades na remoção do code smell são:
- Entender as duas etapas do teste: primeiro obter a ação de fetch gerada pela função, depois executá-la e verificar os argumentos. A auxiliar precisou devolver dois valores (a ação e o caminho da pasta).

Usei o CATSPY para:
- Reaplicar a refatoração usando o mesmo padrão já adaptado no caso 5.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Extract Method: a etapa que obtém a ação de fetch foi movida para _capture_fetch_action. O teste ficou apenas com a execução da ação e as verificações.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
- **Verbose Test (commit a461ebf)**
- em tests/unit/integration/test_hook_integrator.py, método test_root_local_healer_preserves_dependency_source_entries (linha 4431).

Minhas principais dificuldades na remoção do code smell são:
- Aprender a sintaxe {**dicionario, “chave”: valor} do Python, que cria um dicionário novo a partir de outro sem alterá-lo, que até então para mim era nova.

Usei o CATSPY para:
- Reaplicar a refatoração usando o mesmo padrão já adaptado no caso 6.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Extract Variable (remoção de duplicação): o hook repetido foi definido uma única vez em pre_push_matcher e reaproveitado no hook_data e na entrada com _apm_source.

3. Unknown Test

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Unknown Test (commit 14427e51)**
- em tests/unit/install/test_mcp_conflicts.py, método test_all_defaults_no_error (linha 49).

Minhas principais dificuldades na remoção do code smell são:

- A função validate_mcp_conflicts só lança erro ou devolve None, então não há um estado para verificar como nos exemplos do catálogo.
- A única asserção possível é fraca (confirmar que devolve None).

Usei o CATSPY para:

- Entender a definição do Unknown Test (teste sem asserções explícitas, que passa se nenhuma exceção for lançada)
- E a refatoração indicada: introduzir uma asserção que valide o resultado esperado. Nos exemplos do catálogo o teste altera um estado e a asserção confere essa alteração, mas aqui não há estado para conferir, então adaptei a ideia usando o valor devolvido pela função.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Introduzir asserção. O helper _call passou a devolver o resultado da função, e o teste ganhou assert _call() is None.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Unknown Test (commit 1959ea6c)**
- em tests/unit/install/test_mcp_conflicts.py, método test_mcp_name_with_transport_stdio_no_url (linha 54).

Minhas principais dificuldades na remoção do code smell são:

- A mesma limitação do caso anterior: a função só devolve None, então a asserção possível é fraca.

Usei o CATSPY para:

- Reaplicar a refatoração da página do Unknown Test (introduzir uma asserção que
- valide o resultado esperado), usando o mesmo padrão já adaptado no caso anterior.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Introduzir asserção: assert _call(transport="stdio") is None.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Unknown Test(commit 990e1d10)**
- em tests/unit/install/test_mcp_conflicts.py, método test_mcp_name_with_command_argv.

Minhas principais dificuldades na remoção do code smell são:

- Novamente a função validate_mcp_conflicts só devolve None ou lança erro, então a única asserção possível é fraca. Não há estado para verificar.

Usei o CATSPY para:

- Aplicar a refatoração indicada na página do Unknown Test (introduzir asserção), no mesmo molde dos casos anteriores deste arquivo.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Introduzir asserção: assert _call(command_argv=[“npx”, “my-server”]) is None.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Unknown Test (commit bde14b2b)**
- em tests/unit/install/test_mcp_conflicts.py, método test_headers_with_url.

Minhas principais dificuldades na remoção do code smell são:

- A mesma limitação. Além disso, percebi que vários testes do arquivo seguem o mesmo molde, e refatorar todos deixaria a mesma verificação fraca repetida, por isso decidi diversificar os próximos casos em outros arquivos.

Usei o CATSPY para:

- Aplicar a refatoração da página do Unknown Test (introduzir asserção) e perceber, pelos exemplos do catálogo, que uma asserção mais forte, que confere um estado ou efeito observável, seria melhor. Por isso os próximos casos foram buscados em testes onde isso era possível.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Introduzir asserção: assert _call(url="https://example.com/mcp", headers={“X-Key”: “value”}) is None.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Unknown Test (commit b6d2ae8f)**
- em tests/unit/test_view_command_cli_surface.py, método test_no_project_root_skips_lockfile.

Minhas principais dificuldades na remoção do code smell são:

- O comentário do teste dizia que _lookup_lockfile_ref não deveria ser chamada sem a raiz do projeto, mas nada verificava isso, e o teste não tinha patch dessa função.
- Foi preciso acrescentar o patch para poder observar a chamada, o que mudou a forma do with (para o formato com parênteses usado nos outros testes do arquivo).

Usei o CATSPY para:

- Seguir a refatoração da página do Unknown Test (introduzir uma asserção que valide o resultado esperado), desta vez com uma verificação real, como nos exemplos do catálogo que conferem um efeito, e não só “devolve None”.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Introduzir asserção com mock: patch(“apm_cli.commands.view._lookup_lockfile_ref”) as lookup e lookup.assert_not_called().
- Conferi que ela pode falhar passando project_root=tmp_path: o teste falhou com “Called 1 times”.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Unknown Test (commit aa9dd270)**
- em tests/unit/test_view_command_cli_surface.py, método test_rich_rendering_no_context_files.

Minhas principais dificuldades na remoção do code smell são:

- A saída é desenhada pelo Rich, que pode quebrar linhas ou truncar texto. Por isso usei pedaços curtos da mensagem (“No context” e “No agent”) em vez de frases inteiras.

Usei o CATSPY para:

- Aplicar a refatoração do Unknown Test (introduzir asserções que validem o resultado esperado)
- seguindo os exemplos do catálogo em que o teste passa a conferir o que foi produzido, aqui a saída impressa na tela.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Introduzir asserções sobre a saída capturada com capsys (capsys.readouterr().out). Conferi que elas podem falhar trocando cada trecho por um errado.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Unknown Test (commit d76b7c52)**
- em tests/unit/test_view_command_cli_surface.py, método test_import_error_fallback_text.

Minhas principais dificuldades na remoção do code smell são:

- O teste não dizia o que esperar da saída do fallback em texto puro. Foi preciso rodar o teste e ver a saída real para escolher só trechos que existem de fato.

Usei o CATSPY para:

- Aplicar a refatoração do Unknown Test, introduzindo asserções com sentido sobre o resultado, em vez de uma verificação vazia, no mesmo espírito dos exemplos do catálogo (conferir o estado esperado depois da operação).

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Introduzir asserções sobre a saída capturada com capsys: o nome do pacote, o ref, o commit e as contagens de skills, workflows e hooks.
- Cada trecho foi testado trocando por um valor errado, e o teste falhou em todos.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Unknown Test (commit 7eda87af)**
- em tests/unit/install/test_cache_pin.py, método test_write_marker_silently_skips_missing_dir.

Minhas principais dificuldades na remoção do code smell são:

- O teste só chamava a função e confiava em “não levantou exceção”.
- Foi preciso ler o código da função para entender o que ela deveria evitar: criar o diretório que não existe.

Usei o CATSPY para:

- Aplicar a refatoração do Unknown Test (introduzir uma asserção que valide o resultado esperado). Como no exemplo do catálogo em que se confere o estado depois da operação, aqui conferi o estado do sistema de arquivos.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Introduzir asserção sobre o sistema de arquivos: guardei o caminho inexistente numa variável e acrescentei assert not missing.exists(). Conferi que a asserção pode falhar trocando temporariamente a guarda da função por mkdir.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Unknown Test (commit 9251edd8)**
- em tests/unit/integration/test_mcp_integrator.py, método test_noop_for_unknown_server.

Minhas principais dificuldades na remoção do code smell são:

- O teste se chama “noop”, mas nada verificava que realmente não houve efeito.
- Precisei confirmar no código que o cache começa vazio e que a função retorna cedo.

Usei o CATSPY para:

- Aplicar a refatoração do Unknown Test.
- A ideia do catálogo de validar o estado esperado se encaixou bem: se a função não deve fazer nada, o estado do cache deve continuar igual.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Introduzir asserção sobre o estado: assert cache == {}, que garante que nenhuma entrada foi criada ou alterada.
- Conferi que ela falha ao trocar o get por setdefault na função.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Unknown Test (commit c5b1673f)**
- em tests/unit/core/test_auth_phase3.py, método test_empty_host_is_noop.

Minhas principais dificuldades na remoção do code smell são:

- Com ctx=None a função já retornaria mais adiante, então o teste não distinguia o caminho do host vazio.
- Foi preciso montar um ctx válido e um logger em modo verbose para que só a guarda do host vazio impedisse a saída, e validar que o patch do _rich_echo realmente intercepta a chamada (com host preenchido, ele foi chamado).

Usei o CATSPY para:

- Aplicar a refatoração do Unknown Test (introduzir asserção).
- O catálogo mostra asserções sobre estado ou valores, mas aqui o efeito esperado é a ausência de uma chamada, então adaptei a ideia usando um mock.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Introduzir asserção com mock: patch(“apm_cli.utils.console._rich_echo”) as echo e echo.assert_not_called(). Conferi que ela falha ao remover a guarda de host vazio da função.

4. Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Conditional Test Logic (commit d7216a04)**
- em tests/unit/test_auth.py, método test_ado_git_env_strips_raw_github_token_sources.

Minhas principais dificuldades na remoção do code smell são:

- O filtro com if estava dentro de uma comprehension no corpo do teste e se repetia em outro teste.
- Foi preciso garantir que o filtro e as asserções continuassem iguais.

Usei o CATSPY para:

- Entender a definição do Conditional Test Logic (instruções condicionais ou laços dentro do teste) e a refatoração indicada: extrair a lógica condicional para métodos separados.
- Apliquei exatamente essa estratégia, movendo o filtro para uma função auxiliar.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extrair método: criei o helper de módulo _basic_auth_headers(env), e o teste manteve as asserções len == 1 e o valor esperado “:ado-pat”. Conferi que a asserção pode falhar trocando o valor esperado.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Conditional Test Logic (commit a0928409)**
- em tests/unit/test_auth.py, método test_gitlab_git_env_uses_safe_basic_header.

Minhas principais dificuldades na remoção do code smell são:

- A mesma comprehension estava copiada do teste anterior. A dificuldade foi reaproveitar o helper sem mudar o valor esperado de cada teste.

Usei o CATSPY para:

- Aplicar de novo a estratégia da página do Conditional Test Logic (extrair a lógica condicional para um método separado), reaproveitando o helper criado no caso anterior.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extrair método: reutilizei _basic_auth_headers(env), mantendo o valor esperado “oauth2:glpat_fresh”. Conferi que a asserção pode falhar trocando esse valor.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Conditional Test Logic (commit 6e2b95b4)**
- em tests/unit/test_auth.py, método test_concurrent_same_host_emits_once.

Minhas principais dificuldades na remoção do code smell são:

- O laço que enviava as 64 chamadas e depois coletava os resultados precisava continuar relançando exceções das threads. Usei pool.map dentro de list(...), que também propaga as exceções.

Usei o CATSPY para:

- Entender o smell e a ideia de simplificar a lógica no corpo do teste.
- O catálogo só mostra casos de if/else, não laços de concorrência, então a estratégia não se aplicava diretamente.
- Adaptei a ideia substituindo o laço por uma chamada única.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Substituir o laço por uma chamada única: list(pool.map(resolver.emit_stale_pat_diagnostic, [“dev.azure.com”] * 64)).
- Conferi que o teste falha ao trocar call_count == 2 por == 3.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Conditional Test Logic (commit 5072a1b0)**
- em tests/unit/compilation/test_coverage_guarantees.py, método test_hierarchical_coverage_verification.

Minhas principais dificuldades na remoção do code smell são:

- Só trocar o for por parametrize não bastava: o detector continuou acusando os for e if aninhados que calculavam as instruções acessíveis.
- Foi preciso levar esse cálculo para um helper e usar caminhos relativos nos parâmetros, porque a fixture é um diretório temporário.
- A primeira tentativa não foi commitada porque o detector ainda marcava o teste.

Usei o CATSPY para:

- Seguir a refatoração da página do Conditional Test Logic: separar cada caso em um teste próprio e extrair a lógica condicional para um método.
- O catálogo mostra isso com if/else e testes escritos à mão.
- Aqui o smell era um for sobre 5 casos, então adaptei: o @pytest.mark.parametrize faz o papel de “um teste por caso”, e o cálculo interno foi para um helper.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Parametrização do teste (os 5 casos do for viraram parâmetros) e extração de método (_accessible_instruction_names).
- O arquivo passou de 5 para 9 testes. Conferi que o teste falha ao trocar um valor esperado.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Conditional Test Logic (commit af13210b)**
- em tests/unit/compilation/test_coverage_guarantees.py, método test_coverage_constraint_priority_over_efficiency.

Minhas principais dificuldades na remoção do code smell são:

- O teste tinha um if exists() que ignorava em silêncio um arquivo inexistente.
- Antes de removê-lo, li a fixture complex_project e confirmei que ela sempre cria os três arquivos, então a verificação do teste não muda (e passa a falhar se algum faltar).

Usei o CATSPY para:

- Aplicar a estratégia da página do Conditional Test Logic: o catálogo separa os casos em testes distintos e retira a condicional do teste.
- Fiz o equivalente, com um caso parametrizado por arquivo, a condicional removida e a lógica de busca em um método auxiliar.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Parametrização dos 3 caminhos, remoção do if exists() e extração de método (_has_instruction_coverage, com any() no lugar do for/break).
- O arquivo passou de 9 para 11 testes. Conferi que os 3 casos falham ao trocar o nome da instrução esperada.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Conditional Test Logic (commit 9d615ff7)**
- em tests/unit/integration/test_data_driven_dispatch.py, método test_skills_is_multi_target.

Minhas principais dificuldades na remoção do code smell são:

- Dividir o teste em dois não bastava, porque o for passou para o novo teste e o detector continuou acusando.
- Foi preciso parametrizar sobre os 5 nomes. Também foi necessário importar pytest no arquivo.

Usei o CATSPY para:

- Seguir a refatoração do catálogo de dividir o teste em casos separados: o teste de skills ficou isolado, como nos exemplos do catálogo.
- Para os outros 5 casos, que vinham de um for, usei parametrize, que gera um teste por caso.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Dividir o teste e parametrizar: test_skills_is_multi_target ficou só com a asserção de skills, e o novo test_other_primitives_are_single_target é parametrizado com os 5 nomes (prompts, agents, commands, instructions e hooks).
- O arquivo passou de 57 para 62 testes. Conferi que os 5 casos falham ao trocar is False por is True.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Conditional Test Logic (commit 50b8dc1f)**
- em tests/unit/integration/test_data_driven_dispatch.py, método test_dispatch_entries_have_valid_methods.

Minhas principais dificuldades na remoção do code smell são:

- O for ficava no corpo do teste. Foi preciso importar get_dispatch_table no topo do módulo para usá-la no decorador, e conferir que os dois hasattr continuavam os mesmos.

Usei o CATSPY para:

- Aplicar a estratégia da página do Conditional Test Logic de um teste por caso.
- O catálogo faz isso escrevendo testes separados para os ramos de um if/else. Aqui o smell era um for sobre as entradas da tabela, então usei parametrize, que cria um teste por entrada.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Parametrização: parametrize(“name”, sorted(get_dispatch_table())).
- O arquivo passou de 62 para 68 testes e o total de ocorrências do arquivo caiu de 27 para 26. Conferi que os 7 casos falham ao inverter a verificação.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Conditional Test Logic (commit ff5460be)**
- em tests/unit/integration/test_hook_integrator.py, método test_filter_generic_hooks_universal.

Minhas principais dificuldades na remoção do code smell são:

- O teste tinha dois laços. Uma opção era escrever os arquivos com uma list comprehension só pelo efeito colateral, mas isso não é boa prática.
- Optei por escrever os dois arquivos de forma explícita.

Usei o CATSPY para:

- Seguir a refatoração da página do Conditional Test Logic (retirar a lógica condicional do corpo do teste e separar os casos).
- Parametrizei os 5 targets e deixei a escrita dos arquivos explícita, para o teste continuar legível.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Parametrização dos 5 targets e escrita explícita dos arquivos (hooks_file.write_text e telemetry_file.write_text).
- O arquivo passou de 188 para 192 testes e o total de ocorrências caiu de 29 para 27. Conferi que os 5 casos falham ao inverter a asserção.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Conditional Test Logic (commit 69c49fe7)**
- em tests/unit/core/test_public_github_anonymous_first.py, método test_private_github_file_fetch_uses_one_scoped_fallback.

Minhas principais dificuldades na remoção do code smell são:

- O for com if e pop que montava o ambiente sem credenciais estava no corpo do teste e se repetia em outro teste.
- Foi preciso manter exatamente a mesma limpeza de variáveis.

Usei o CATSPY para:

- Aplicar a estratégia da página do Conditional Test Logic: extrair a lógica condicional para métodos separados.
- O laço com if que montava o ambiente foi para funções auxiliares, e o corpo do teste ficou só com a preparação e as asserções.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extrair método: _poisoned_environment_without_credentials() e _is_github_credential().
- O total de ocorrências do arquivo caiu de 23 para 21, e o teste saiu da lista do detector.
- Conferi que o teste falha quando o helper deixa de remover as credenciais.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

- **Conditional Test Logic (commit 5b6f8f0a)**
- em tests/unit/integration/test_data_driven_dispatch.py, método test_every_target_primitive_has_dispatch_path.

Minhas principais dificuldades na remoção do code smell são:

- O teste tinha dois laços aninhados (target e primitiva) e verificava tudo numa única execução, então uma falha não dizia qual combinação estava errada.
- A dificuldade foi montar os pares no decorador mantendo exatamente as mesmas combinações que o laço percorria (46) e conferir que a contagem batia com a soma das primitivas de cada target.

Usei o CATSPY para:

- Seguir a ideia da página do Conditional Test Logic de que cada caso deve ser verificado separadamente, para que uma falha indique qual caso está errado.
- Como o catálogo trata de if/else e aqui eram laços aninhados, usei parametrize sobre os pares (target, primitiva), gerando um teste por combinação.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Parametrização: @pytest.mark.parametrize sobre os pares (target, primitiva), com a mesma asserção de antes.
- O arquivo passou de 68 para 113 testes, e o total de ocorrências do arquivo caiu de 26 para 24. Conferi que os 46 casos falham ao inverter a asserção.